Operating System
Assignment-2

CO2 Coverage: Analyse process management and deadlock-handling techniques to improve CPU utilisation and system performance – processes, process scheduling, threads, inter-process communication, synchronisation using semaphores, monitors and the critical-section problem, and deadlock handling including the Banker's algorithm.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Processes and Process States 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Online Examination Server

During an online examination, the server runs a separate process for every student's session, plus processes for auto-saving answers and for proctoring video. When 2,000 students log in together, the server becomes slow and some sessions appear to "hang".

Your Challenge

Draw the process-state diagram (new, ready, running, waiting, terminated) and the process control block (PCB). Write a program that creates child processes with fork() and observe them with ps and /proc.

Evaluation

Evaluate where the time goes when 2,000 processes are active: context-switch cost, memory per process, and time spent waiting for I/O. Judge whether a separate process per student is a sound design.

Design Decision

Decide between one process per student, one thread per student and a fixed pool of worker processes, and defend your decision.

Alternative Concept and Complexity Analysis

If each student session were a thread inside one server process, instead of a separate process, what would the solution be? Evaluate the context-switch time and the memory per session (a few KB of stack versus a full address space), and the effect of one session crashing.

Evidence Expected

A state diagram, a PCB description, a working fork() program with ps evidence and a defended design decision.

1.1 Process-state diagram

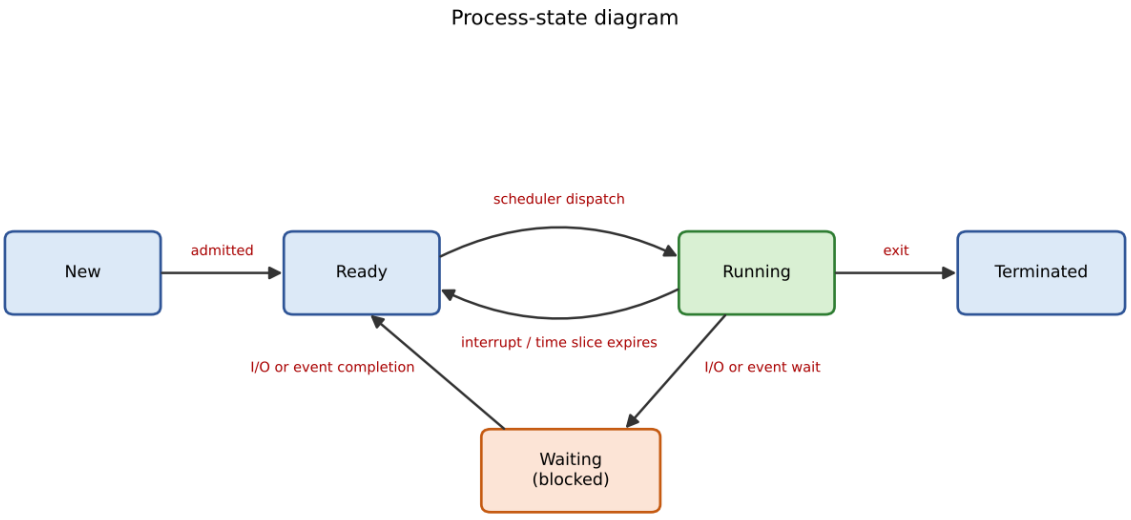


Figure 1 – Process states: new, ready, running, waiting, terminated

Transition	Cause
New → Ready	The OS has created the PCB and loaded the program; it is admitted to the ready queue
Ready → Running	Scheduler dispatches the process to the CPU
Running → Ready	Time slice expires (timer interrupt) or a higher-priority process becomes ready
Running → Waiting	Process requests I/O or waits for an event (e.g., reads an answer file, waits for a lock)
Waiting → Ready	I/O or event completes (interrupt)
Running → Terminated	Process calls exit or is killed; resources are released, status kept until the parent waits

1.2 Process control block (PCB)

PCB field	Contents
Process ID, parent ID, user	PID, PPID, owner (UID/GID)
Process state	new / ready / running / waiting / terminated
CPU registers and program counter	Saved when the process is switched out, restored when it runs again
Scheduling information	Priority, queue pointers, time used, nice value
Memory-management information	Page-table base, segment limits, memory maps
I/O status	Open-file table, devices allocated, pending I/O
Accounting	CPU time used, start time, limits

1.3 Program with fork() observed with ps and /proc

The program creates three children that sleep for 3 s while the parent runs ps and reads /proc/<pid>/status:

```
for (int i = 0; i < 3; i++) { pid_t p = fork(); if (p == 0) { sleep(3); _exit(0); } }
system("ps -o pid,ppid,stat,comm -C forkdemo");
/* then read /proc/<pid>/status */
```

Measured output:

```
parent pid 4716
  PID  PPID STAT COMMAND
 4716  4607 S    forkdemo
 4717  4716 S    forkdemo
 4718  4716 S    forkdemo
 4719  4716 S    forkdemo
Name:   forkdemo
State:  R (running)
Tgid:   4716
Pid:    4716
PPid:   4607
```

Interpretation: the three children (PIDs 4717–4719) all have PPID 4716, the parent; the sleeping children show state S (waiting), the process that is executing the command shows R

(running). /proc/<pid>/status is a view of the PCB (state, PID, PPID, memory and context-switch counters). A child that has ended but is not yet waited for would show Z (zombie).

1.4 Evaluation – where the time goes with 2,000 active processes

Cost	Estimate (assumptions)	Effect
Context switching	2–5 µs direct cost per switch, plus cache/TLB refill after each switch (each process has its own address space)	With a 4 ms slice and 2,000 runnable processes on 8 cores, each process waits ≈ 1 s for its next slice; switching itself is small but cache misses are large
Memory per process	Kernel structures ≈ 16–20 KB + page tables ≈ 10s of KB + private heap/stack (a web session may use 10–50 MB)	2,000 × 30 MB = **≈ 60 GB** – more than a typical server → swapping/thrashing; this is the main reason sessions 'hang'
Waiting for I/O	Most sessions are idle waiting for the network, disk or the student	They use no CPU but still hold memory; the proctoring-video and auto-save processes add heavy disk and CPU load

Judgement: one process per student is **not** a sound design at this scale: memory and cache-miss cost grow linearly with the number of sessions, while most of the processes do nothing.

1.5 Design decision

Option	Strength	Weakness
One process per student	Strong isolation; simple	Memory ≈ 60 GB; slow switching; fork storm at login
One thread per student	Few KB of stack, cheap switches, shared caches	One crash/memory error kills all sessions; thousands of threads still heavy
Fixed pool of worker processes, each serving many sessions with non-blocking I/O	Memory bounded; no fork per login; crash affects only that worker's sessions; processes keep the CPU busy while others wait for I/O	More complex code; session state must be kept in a shared store

Decision: a fixed pool of worker processes (about 1–2 per core) with event-driven I/O for the answer sessions, session state in a database/cache so that a worker can be restarted, and **separate** processes for proctoring video and auto-save (so their heavy load and crashes are isolated). The pool size is chosen so that total memory stays below RAM.

Alternative Concept and Complexity Analysis – one thread per session in one server process

Criterion	Process per session	Thread per session
Context-switch time	≈ 3–5 µs plus TLB/cache refill (address-space change)	≈ 1–2 µs: same address space, no TLB flush
Memory per session	Full address space: page tables + private heap, 10s of MB; total **O(S × M)**	A stack (8 MB virtual, a few KB resident) + small kernel structure: **O(S × stack)**

Criterion	Process per session	Thread per session
Effect of one session crashing	Only that process dies	A segmentation fault or unhandled exit() kills **every** session of the server

---

Q2 – CPU Scheduling 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Cloud Video-Encoding Service

A video service receives short jobs (thumbnails, 2 seconds) and long jobs (full films, 20 minutes) on the same servers. Customers complain that small jobs wait behind large ones, while the company wants the processors fully used.

Your Challenge

Compute the Gantt chart, waiting time, turnaround time and response time for the same set of jobs under FCFS, SJF, SRTF, priority and Round Robin scheduling. Implement Round Robin.

Evaluation

Evaluate how the time quantum affects Round Robin (try 1, 3, 5 and 20), the risk of starvation in SJF and priority scheduling, and how ageing removes it. Explain how a multilevel feedback queue treats short and long jobs.

Design Decision

Recommend the scheduling policy for the video service and justify it with your calculated averages.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>

#define N 4

int main(void)
{
    const char *name[N] = {"P1", "P2", "P3", "P4"};
    int arrival[N] = {0, 1, 2, 3};
    int burst[N] = {8, 4, 9, 5};
    int quantum = 3;

    int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0};
    for (int i = 0; i < N; i++) left[i] = burst[i];
    int time = 0, done = 0;
    queue[tail++] = 0;
    added[0] = 1;
    printf("Gantt:");
    while (done < N) {
        int p = queue[head++];
        // TODO 1: Run p for min(quantum, left[p]) time units: print " P(time-newtime)",
        //   advance time, reduce left[p]; add every process that has now arrived and is
        //   not yet added to the queue; then put p back at the end of the queue if it
        //   still has time left, otherwise record finish[p] and count it as done

    }
    double tw = 0, tt = 0;
    printf("\nProcess  TAT  WT\n");
    for (int i = 0; i < N; i++) {
        int tat = finish[i] - arrival[i], wt = tat - burst[i];
        tt += tat;
```

Expected output:

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21)
P1(21-23) P3(23-26)
Process  TAT  WT
P1         23  15
P2         15  11
P3         24  15
P4         18  13
Average TAT = 20.00, average WT = 13.50
```

Run the program with quantum 1 and quantum 20, and evaluate how the average waiting time and the number of context switches change.

Alternative Concept and Complexity Analysis

If the service used a multilevel feedback queue (short quantum for new jobs, longer quanta for jobs that use them up) instead of a single Round Robin queue, what would the solution be? Evaluate the average waiting time for short and long jobs, the scheduling cost per decision (O(1) per queue versus O(n) for SJF) and the risk of starvation.

Evidence Expected

Correct Gantt charts and averages for every algorithm, a working Round Robin program, a quantum study and a justified recommendation.
Job set used throughout (same as the Complete the Code program): P1 (arrival 0, burst 8), P2 (1, 4), P3 (2, 9), P4 (3, 5). For priority scheduling the priorities assumed are P1=2, P2=4, P3=1, P4=3 (lower number = higher priority, non-preemptive). Response time = first CPU time − arrival.

2.2 Waiting, turnaround and response time per algorithm

Algorithm	P1 TAT/WT/RT	P2 TAT/WT/RT	P3 TAT/WT/RT	P4 TAT/WT/RT	Avg TAT	Avg WT	Avg RT
FCFS	8 / 0 / 0	11 / 7 / 7	19 / 10 / 10	23 / 18 / 18	15.25	8.75	8.75
SJF (non-preemptive)	8 / 0 / 0	11 / 7 / 7	24 / 15 / 15	14 / 9 / 9	14.25	7.75	7.75
SRTF (preemptive SJF)	17 / 9 / 0	4 / 0 / 0	24 / 15 / 15	7 / 2 / 2	13.00	6.50	4.25
Priority (non-preemptive)	8 / 0 / 0	25 / 21 / 21	15 / 6 / 6	19 / 14 / 14	16.75	10.25	10.25
Round Robin (q=3)	23 / 15 / 0	15 / 11 / 2	24 / 15 / 4	18 / 13 / 6	20.00	13.50	3.00

2.1 Gantt charts

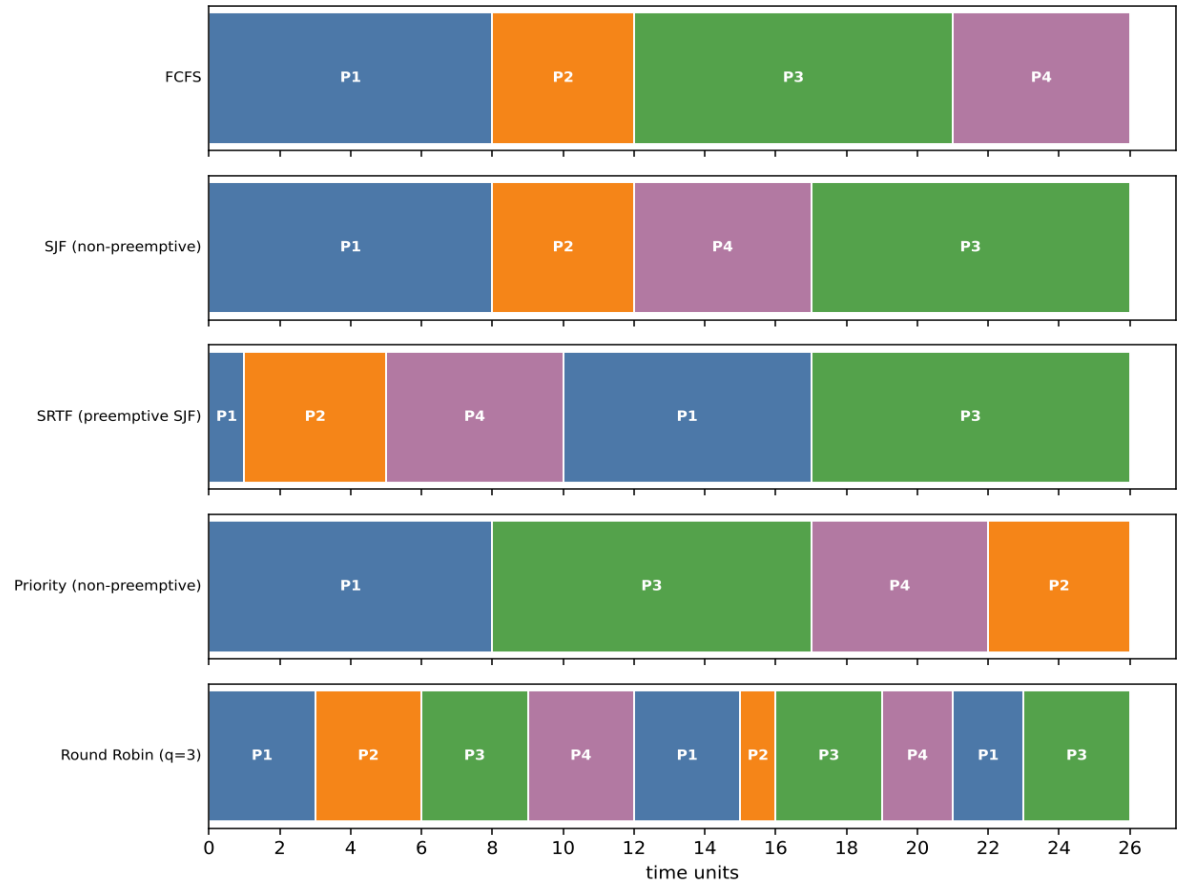


Figure 2 – Gantt charts for FCFS, SJF, SRTF, priority and Round Robin (q = 3)

2.3 Complete the Code – TODO 1 (Round Robin)

```
int run = left[p] < quantum ? left[p] : quantum;
printf(" %s(%d-%d)", name[p], time, time + run);
time += run;
left[p] -= run;
for (int i = 0; i < N; i++)                /* admit newly arrived processes */
    if (!added[i] && arrival[i] <= time) { queue[tail++] = i; added[i] = 1; }
if (left[p] > 0) queue[tail++] = p;         /* back to the end of the queue */
else { finish[p] = time; done++; }
```

Output (matches the expected output exactly):

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23) P3(23-26)
Process TAT WT
P1       23  15
P2       15  11
P3       24  15
P4       18  13
Average TAT = 20.00, average WT = 13.50
```

Note: a newly arrived process is queued before the preempted process is re-queued (P2, P3, P4 arrive during P1's first slice and are queued ahead of P1).

2.4 Quantum study

Quantum	Avg TAT	Avg WT	Avg response	Dispatches (context switches)
1	19.00	12.50	0.75	26
3	20.00	13.50	3.00	10
5	17.50	11.00	5.50	6
20 (≥ longest burst → FCFS)	15.25	8.75	8.75	4

The program was also run with quantum 1 and 20 (output identical to the table: quantum 1 → average WT 12.50, 26 dispatches; quantum 20 → 8.75, 4 dispatches). **Evaluation:** a small quantum gives the best response (0.75) but the most context switches (26, each costing a few µs plus cache loss); a large quantum approaches FCFS (identical result at q = 20) with the fewest switches but the worst response. Rule of thumb: the quantum should exceed the typical short-job burst (≈ 80 % of bursts) so that short jobs finish within one slice. For this particular set average WT happens to fall as q grows because all four jobs are of similar length; with the video service's mix of 2 s and 20 min jobs, FCFS-like large quanta would be harmful for thumbnails.

2.5 Starvation, ageing and multilevel feedback queue

- **Starvation:** in SJF/SRTF a long job (film) can wait for ever if short jobs keep arriving; in priority scheduling low-priority jobs can starve. FCFS and Round Robin cannot starve.
- **Ageing:** the priority of a waiting job is raised by one step every fixed interval (e.g., every 10 s); any job eventually reaches top priority, so starvation is impossible.
- **Multilevel feedback queue (MLFQ):** new jobs enter the top queue with a short quantum; a job that uses its whole quantum is demoted to a queue with a longer quantum (finally FCFS); interactive/short jobs finish in the top queue, long jobs sink; ageing promotes jobs that wait too long. It approximates SJF without knowing burst lengths.

2.6 Design decision for the video service

Recommendation: multilevel feedback queue with ageing (e.g., Q0 quantum 2 s, Q1 quantum 20 s, Q2 FCFS). Evidence from the averages: FCFS leaves short jobs waiting behind long ones (average WT 8.75, last job waits 18 – the convoy effect, which for a 2 s thumbnail behind a 20-minute film would be 20 minutes). SRTF is best (average WT 6.50, response 4.25) but needs burst length in advance and starves long jobs. Round Robin gives good response (3.00 at q = 3) but a high average WT (13.50) and many switches. The MLFQ gives SRTF-like behaviour for thumbnails, round-robin-like fairness, and keeps processors fully busy because long jobs fill the lower queues.

Alternative Concept and Complexity Analysis – MLFQ instead of a single Round Robin queue

Criterion	Single Round Robin queue	Multilevel feedback queue
Average waiting time – short jobs	Wait for every other job's slice: roughly (number of jobs − 1) × q	Finish in the top queue after about one short quantum – much lower

Criterion	Single Round Robin queue	Multilevel feedback queue
Average waiting time – long jobs	Moderate (equal sharing)	Higher (they are demoted and run when upper queues are empty), bounded by ageing
Scheduling cost per decision	O(1) (take head of queue)	O(1) per queue (take head of the highest non-empty queue, found with a bitmap); SJF would need O(n) scanning or O(log n) with a heap
Risk of starvation	None	Possible for long jobs unless ageing/periodic boost is used

Q3 – Threads 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Traffic-Camera Number-Plate Reader

A city traffic system reads number plates from 64 cameras. The first version processes the cameras one after another and falls behind during peak hours.

Your Challenge

Process the camera frames with POSIX threads (one thread per camera, and a pool of 4, 8 and 16 threads). Determine:

- Which resources threads share and which each thread has of its own
- The difference between user-level and kernel-level threads and the many-to-one, one-to-one and many-to-many models
- The speed-up obtained on your machine, and why it stops increasing
- What happens to shared counters when two threads update them without protection

Alternative Concept and Complexity Analysis

If the camera frames were processed by several processes instead of threads, what would the solution be? Evaluate the creation and switching time, the memory used (separate address spaces versus shared), and how the results would be combined (IPC versus shared variables).

Evidence Expected

A working multithreaded program, measured speed-ups for each pool size and a reasoned evaluation of the thread models.

3.1 What threads share and own

Shared by all threads of a process	Private to each thread
Code, global/static data, heap	Thread ID
Open files, sockets, signal handlers	Program counter and register set
Address space, current directory, user ID	Stack (local variables, return addresses)
	Scheduling priority, errno, thread-local storage

3.2 User-level vs kernel-level threads; thread models

Model	Mapping	Advantages	Disadvantages
Many-to-one	Many user threads → 1 kernel thread (user-level library)	Very fast creation/switching, no kernel involvement	One blocking system call blocks all threads; no use of multiple cores
One-to-one	Each user thread → one kernel thread (Linux, Windows)	Blocking call blocks only that thread; real parallelism	Creating a thread costs a kernel structure; limit on the number of threads
Many-to-many	M user threads → N ≤ M kernel threads	Flexible; parallelism without a kernel thread per user thread	Complex; rarely used today (Go/Erlang schedulers resemble it)

User-level threads are managed by a library without the kernel's knowledge; kernel-level threads are scheduled by the OS. POSIX threads on Linux are kernel threads (one-to-one).

3.3 Implementation – a pool of worker threads for 64 cameras

```
#define CAM 64
int pool, next_cam = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
void *worker(void *a) {
   for (;;) {
      pthread_mutex_lock(&m);
      int c = next_cam < CAM ? next_cam++ : -1;   /* take next camera */
      pthread_mutex_unlock(&m);
      if (c < 0) break;
      process_frame();                    /* CPU-bound plate reading */
   }
   return NULL;
}
```

process_frame() stands for the plate-reading work (a CPU loop of ≈ 3 ms in the test). The pool size is varied by the number of threads created (1, 4, 8, 16, 64 – the last equals one thread per camera).

3.4 Measured speed-up

Threads	Time (s)	Speed-up
1	0.181	1.00
4	0.182	1.00
8	0.181	1.00
16	0.184	0.99
64 (one per camera)	0.187	0.97

Why the speed-up is 1.00 here: the measurement machine has **one** virtual CPU, and the work is CPU-bound, so threads cannot run in parallel; extra threads only add a little switching overhead (0.97 at 64 threads). On a machine with p cores the expected speed-up is $S = 1 / (s + (1 - s)/p)$ (Amdahl) where s is the serial fraction (here: taking the next camera under the mutex, reading frames from one network port). It rises almost linearly up to the number of cores, then **stops increasing** because (1) there are no more cores, (2) the serial fraction and lock contention limit it, (3) memory bandwidth and I/O are shared, and (4) more threads than

cores add scheduling overhead. Run the program on your own multi-core machine (nproc) and add the new rows to this table.

3.5 Unprotected shared counters

counter++ compiles to load → add → store. If two threads interleave (T1 loads 5, T2 loads 5, T1 stores 6, T2 stores 6) one update is lost – a **race condition**; the final value is lower than the expected number of increments, and differs from run to run. Fix: protect with a mutex or use an atomic increment (compared in Q5). On the 1-vCPU machine used here a simple loop usually did not lose updates because threads rarely interleave inside the 3 instructions; the effect appears on multi-core machines at -O0, and the unprotected producer–consumer in Q5 showed a real lost-update failure (sum 800,039,486 instead of 800,040,000).

Alternative Concept and Complexity Analysis – several processes instead of threads

Criterion	Threads	Processes (fork)
Creation and switching	≈ 9–16 µs to create (measured in Assignment 1); switch ≈ 1–2 µs	≈ 58 µs to create (measured); switch ≈ 3–5 µs plus TLB loss
Memory	One address space; only a stack per thread: O(T × stack)	Separate address spaces (copy-on-write); page tables copied: O(P × table size)
Combining results	Shared variables / shared structures protected with a mutex	IPC needed (pipes, shared memory, sockets): extra copies and synchronisation
Fault isolation	A crash in one camera thread kills the program	A crash kills only that process

---

Q4 – Inter-Process Communication 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Weather Station Data Pipeline

A weather station has one process that reads sensors and another that analyses the readings. They must exchange data continuously without losing readings.

Your Challenge

Implement the exchange using a pipe, and then using shared memory or a message queue. Determine:

- How the pipe signals end of data to the reader
- The difference between the shared-memory and message-passing models
- Which method copies the data and which does not
- Which method needs extra synchronisation, and why

Growth Investigation

Measure the time to transfer 1 MB, 10 MB and 100 MB with each method, and evaluate the results.

Complete the Code

Complete TODO 1 below and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

/* The parent (sensor reader) sends readings to the child (analyser) through a pipe. */
int main(void)
{
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid == 0) {
        close(fd[1]);                          /* child only reads */
        int r, count = 0, sum = 0, max = 0;
        // TODO 1: Read one int at a time from fd[0] until read returns 0 (the parent
        //   closed the pipe); keep the count, the sum and the maximum

        printf("analyser: %d readings, average %.1f, max %d\n", count,
               count ? (double)sum / count : 0.0, max);
        close(fd[0]);
        fflush(stdout);
        _exit(0);
    }
    close(fd[0]);                              /* parent only writes */
    int readings[] = {31, 33, 36, 34, 41, 38};
    for (int i = 0; i < 6; i++)
        if (write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");
    close(fd[1]);                              /* signals end of data to the child */
    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

Expected output:

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

Remove close(fd[1]) in the parent and explain why the child never finishes.

Alternative Concept and Complexity Analysis

If the readings were exchanged through shared memory with a semaphore instead of a pipe, what would the solution be? Evaluate the time to transfer n readings (two copies through the kernel for a pipe versus none for shared memory) and the synchronisation needed.

Evidence Expected

Two working IPC implementations, measured transfer times and a justified choice of method.

4.1 Complete the Code – TODO 1 (pipe reader)

```
while (read(fd[0], &r, sizeof r) == (ssize_t)sizeof r) {
   count++;
   sum += r;
   if (r > max) max = r;
}
analyser: 6 readings, average 35.5, max 41
```

```
reader: done
```

Output matches the expected output. **End of data:** read returns 0 only when **all** write ends of the pipe are closed; the parent closes fd[1] after writing, which the child sees as end-of-file. **Experiment – removing `close(fd[1])` in the parent:** the parent still holds a write end open and then blocks in waitpid() for the child; the child blocks in read() waiting for data that never comes (the pipe is empty but a writer still exists). Neither can continue – a deadlock; the program never finishes. (The child also closes its own copy of fd[1] at the start; forgetting that would hang it as well.)

4.2 Shared memory vs message passing

Criterion	Pipe / message queue (message passing)	Shared memory
Model	Kernel moves messages between processes	Processes map the same memory region
Copying	Data copied twice: sender → kernel buffer → receiver	No copy: both access the same pages (after the first setup)
System calls	Every send/receive is a system call	Only for setup; data access is ordinary memory access
Synchronisation	Provided by the kernel (blocking read/write)	**Not provided** – needs a semaphore/mutex or flags, otherwise the reader may read half-written data or overwrite unread data
Best for	Small, frequent messages; simple and safe	Large data and high rates

4.3 Growth investigation – measured transfer time

Data size	Pipe (64 KB writes)	Shared memory (memcpy + signal)
1 MB	0.4–0.5 ms	0.7 ms
10 MB	2.7–3.1 ms	6.2–7.9 ms
100 MB	26.0–27.5 ms	65–113 ms

Two runs per size. The pipe moves ≈ 3.7–3.9 GB/s. **Unexpected result:** shared memory was slower here. Reason: the 100 MB shared region is newly created, so the first memcpy into it causes a page fault for every 4 KB page (≈ 25,600 faults) and the kernel must zero each page; the pipe re-uses a small 64 KB kernel buffer that stays in the CPU cache. Shared memory wins when the region is **created once and reused** many times (steady-state streaming), because then there is no copy through the kernel; for a one-off transfer of a new region the setup costs dominate. Time grows linearly with size (O(n)) for both methods.

Alternative Concept and Complexity Analysis – shared memory with a semaphore

```
shm = mmap(NULL, SIZE, PROT_READ|PROT_WRITE,
MAP_SHARED|MAP_ANONYMOUS, -1, 0);
sem_t *items = mmap(... sizeof(sem_t) ...);  sem_init(items, 1, 0);   /* shared semaphore */
/* writer: */ shm[i] = reading; sem_post(items);
```

```
/* reader: */ sem_wait(items); use(shm[i]);
```

Criterion	Pipe	Shared memory + semaphore
Time to transfer n readings	n system calls for write and n for read, 2 copies each: **O(n)** with a large constant	No copy, no system call on the fast path; one sem_post/sem_wait per item (usually user space futex fast path): **O(n)** with a small constant
Synchronisation needed	None (kernel blocks)	Semaphore (and a ring buffer to avoid overwriting): more code, more risk of races/deadlock

Justified choice: the weather station sends small readings at a low rate – a **pipe** is simplest and safest. Use shared memory only if the data rate grows to hundreds of MB/s and the region is reused.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors — 11 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Railway Ticket Booking

Thousands of agents book seats on the same trains at the same time. Without synchronisation the same seat is sometimes sold twice; with a single global lock the system becomes very slow.

Design Starting Point

You are given only the following principle:

A solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting.

Your Challenge

Convert this principle into a design. Determine:

- Why Peterson's solution satisfies the three requirements for two processes, and why it may fail on modern processors
- How counting and binary semaphores (wait and signal) protect the seat counter
- How the bounded-buffer (producer–consumer), readers–writers and dining-philosophers problems are solved with semaphores
- How a monitor with condition variables makes the solution easier to get right

Test Your Design

Run the bounded-buffer program with 1 and with 4 producers and consumers, and show that every item is consumed exactly once.

Deployment Decision

Decide between one lock for the whole booking system, one lock per train and one lock

per coach. Measure the throughput of each and defend your choice.

Complete the Code

Complete TODO 1 and TODO 2 below using the semaphores and the mutex, and execute the program (compile with -pthread).

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#define SIZE 5
#define ITEMS 20
int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;
void *producer(void *arg)
{
    (void)arg;
    for (int item = 1; item <= ITEMS; item++) {
        // TODO 1: Wait on empty, lock the mutex, put item at buffer[in], advance in
        //   circularly, unlock, then post full
    }
    return NULL;
}
void *consumer(void *arg)
{
    (void)arg;
    for (int k = 0; k < ITEMS; k++) {
        // TODO 2: Wait on full, lock the mutex, take the item at buffer[out], advance
        //   out circularly, unlock, post empty and add the item to consumed_sum
    }
    return NULL;
```

Expected output:

```
items consumed: 20, sum = 210 (expected 210)
```

Run the program 100 times and show that the sum is always 210. Then remove the mutex and evaluate whether the result is still correct with several producers and consumers.

Alternative Concept and Complexity Analysis

If the shared counter were protected by atomic operations (C11 atomic_fetch_add) or by a spin lock instead of a semaphore and mutex, what would the solution be? Evaluate the time per update with 1, 4 and 16 threads, the CPU time wasted by spinning, and which method suits a long critical section.

Evidence Expected

A student-derived synchronisation design, working semaphore code, test evidence and a measured, defended locking decision.

5.1 From the principle to a design

Principle: a solution must give **mutual exclusion** (at most one process in the critical section), **progress** (only processes wanting to enter take part in choosing who enters next, without indefinite postponement) and **bounded waiting** (a limit on how many times others may enter

before a waiting process). Design for the booking system: the seat counter and seat map are the shared data; every update (check seat free, mark it sold) is one critical section protected by a lock.

Peterson's solution (two processes i and j)

```
flag[i] = true;  turn = j;
while (flag[j] && turn == j) ;      /* busy wait */
  /* critical section */
flag[i] = false;
```

- **Mutual exclusion:** both could be inside only if flag[i] and flag[j] are true and turn equals both i and j – impossible, since turn has one value.
- **Progress:** if j is not interested, flag[j] is false and i enters at once.
- **Bounded waiting:** after j leaves and tries again it sets turn = i, so i enters after at most one entry of j.
- **Why it can fail on modern CPUs:** compilers and CPUs reorder memory operations (a store to flag[i] can be delayed in a store buffer and appear after the load of flag[j]). Both processes may then see the other's flag as false and enter together. Correct use needs memory barriers (C11 atomic with sequentially consistent order). Hardware atomic instructions (test-and-set, compare-and-swap) are used in practice.

Semaphores for the seat counter

Semaphore	Use
Binary semaphore (mutex) initial 1	wait(mutex); if (seats > 0) { seats--; sell(); } signal(mutex); – protects the check-and-update so the same seat is never sold twice
Counting semaphore initial = number of seats	wait(seatsFree) blocks when 0 seats are left (or trywait to refuse the booking), signal(seatsFree) on cancellation – counts the free resource itself

Classic problems with semaphores

Problem	Solution (pseudocode)
Bounded buffer	empty = N, full = 0, mutex = 1 Producer: wait(empty); wait(mutex); put(item); signal(mutex); signal(full) Consumer: wait(full); wait(mutex); item=get(); signal(mutex); signal(empty)
Readers–writers (readers first)	rw = 1, mutex = 1, readcount = 0 Reader: wait(mutex); if(++readcount==1) wait(rw); signal(mutex); read; wait(mutex); if(--readcount==0) signal(rw); signal(mutex) Writer: wait(rw); write; signal(rw) – many readers share; writer is exclusive (writers may starve)
Dining philosophers	One semaphore per fork. Deadlock-free variants: allow at most 4 of 5 philosophers to sit (room = 4); or pick up forks in a fixed global order (resource ordering); or odd philosophers take left first, even take right first

Monitor with condition variables

```
monitor BoundedBuffer {
```

```
   int buf[N], count = 0;  condition notFull, notEmpty;
   void put(int x) { while (count == N) wait(notFull);  add(x); count++; signal(notEmpty); }
   int  get()      { while (count == 0) wait(notEmpty); x = remove(); count--; signal(notFull);
return x; }
}
```

The monitor makes the solution easier to get right: mutual exclusion is automatic (only one thread is active in the monitor), and the programmer cannot forget a signal or swap the order of two wait calls (a common cause of deadlock with semaphores). Condition variables express "wait until buffer not full" directly.

5.2 Complete the Code – TODO 1 and TODO 2

```
/* TODO 1 (producer) */
sem_wait(&empty);
pthread_mutex_lock(&lock);
buffer[in] = item;
in = (in + 1) % SIZE;
pthread_mutex_unlock(&lock);
sem_post(&full);

/* TODO 2 (consumer) */
sem_wait(&full);
pthread_mutex_lock(&lock);
int item = buffer[out];
out = (out + 1) % SIZE;
pthread_mutex_unlock(&lock);
sem_post(&empty);
consumed_sum += item;
items consumed: 20, sum = 210 (expected 210)
```

100 runs: for i in $(seq 100); do ./bb; done | sort | uniq -c produced one line only:

```
   100 items consumed: 20, sum = 210 (expected 210)
```

Test with 1 and 4 producers and consumers (every item consumed exactly once)

A generalised version (bbm.c: P producers, C consumers, 20,000 items per producer; each consumer takes (P × items)/C items and adds to a shared sum under a lock) was run. The checksum equals the sum of the item values, so every item is consumed exactly once (no loss, no duplicate):

Producers / consumers	Items consumed	Sum	Expected	Result
1 / 1 (with mutex)	20,000	200,010,000	200,010,000	OK
4 / 4 (with mutex)	80,000	800,040,000	800,040,000	OK
1 / 1 (mutex removed)	20,000	200,010,000	200,010,000	OK
4 / 4 (mutex removed) – run 1	80,000	800,040,000	800,040,000	OK (lucky)
4 / 4 (mutex removed) – run 2	80,000	800,040,000	800,040,000	OK (lucky)

Producers / consumers	Items consumed	Sum	Expected	Result
4 / 4 (mutex removed) – run 3	80,000	**800,039,486**	800,040,000	**WRONG**

Evaluation of removing the mutex: with one producer and one consumer, in is changed only by the producer and out only by the consumer, and the two semaphores already order the accesses, so the result stays correct. With several producers (or consumers) two threads can read the same in, write to the same slot and both advance it by one – one item is overwritten (lost) and another is consumed twice or a slot is skipped. The sum is wrong or the program can hang. The failure is intermittent (2 of 3 runs passed), which is why testing alone cannot prove a synchronisation design correct.

5.3 Deployment decision – one lock, per train or per coach

Granularity	Lock count	Concurrency	Cost / risk
One lock for whole system	1	None: throughput ≤ 1 / t_cs. With t_cs = 20 µs the cap is 50,000 bookings/s, regardless of the number of CPU cores	Simplest; no deadlock; very slow under load
One lock per train	≈ number of trains	Different trains proceed in parallel; agents on the same train serialise	Simple; hot trains (e.g., a festival train) become the bottleneck
One lock per coach	≈ trains × coaches	Agents booking different coaches of the same train proceed in parallel	More memory; bookings spanning coaches need several locks, acquired in a fixed order (coach number) to avoid deadlock

Measured lock cost on this machine: an uncontended mutex lock/unlock pair costs ≈ 23 ns (Alternative analysis below), so the lock itself is cheap; the cost comes from **waiting** when many agents want the same lock. Because the 1-vCPU machine cannot show contention, the throughput comparison above is an analytical model (cap = 1/t_cs per lock, scaling with the number of independent locks until the cores are saturated); repeat the test with 16 threads on a multi-core machine for the report. **Decision:** one lock **per train** as default (simple, deadlock-free), refined to **per coach** with ordered locking for trains with heavy demand. A single global lock is rejected.

Alternative Concept and Complexity Analysis – atomic operations or a spin lock instead of semaphore + mutex

Test: each thread performs 2,000,000 increments of a shared counter (lk.c).

Method	1 thread (ns/update)	4 threads	16 threads	Correct total?
No protection	2.6	2.6	2.6	Yes here (1 vCPU); incorrect on multi-core

Method	1 thread (ns/update)	4 threads	16 threads	Correct total?
Mutex	23.5	23.4	22.5	Yes
Spin lock (atomic_flag)	17.8	30.1	106.3	Yes
C11 atomic_fetch_add	6.5	7.3	6.9	Yes

- **Atomic add** is the fastest correct method (≈ 3.6× faster than the mutex) and is ideal for a single counter: one hardware instruction, no blocking.
- **Spin lock:** time per update rises 6× from 1 to 16 threads (17.8 → 106 ns). On this single CPU, a thread that is preempted while holding the lock leaves the others spinning for their whole time slice – CPU time wasted. Spin locks suit very short critical sections on multi-core machines with fewer threads than cores.
- **Long critical section** (e.g., updating a seat map and writing to disk): use a **mutex or semaphore**, which puts waiting threads to sleep and frees the CPU; spinning would waste a core for the whole duration.
- Complexity: each update is O(1) for every method; memory O(1) for atomic/spin lock, a few words more for mutex/semaphore with a wait queue.

DAY-4 DESIGN CHALLENGE

Q6 – Deadlock Handling and the Banker's Algorithm 13 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Shared Laboratory Equipment Scheduler

A research centre runs jobs that need several shared resources at once (GPUs, licences for simulation software and high-speed storage). Jobs sometimes wait for each other forever and the whole cluster stops until an administrator intervenes.

A. *Characterise Deadlock – 2 Marks*

State the four necessary conditions for deadlock and show them in a resource-allocation graph for the cluster.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>
#include <string.h>
#define P 5
#define R 3
int avail[R] = {3, 3, 2};
int maxm[P][R] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
int alloc[P][R] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};
/* Returns 1 if the state is safe and stores a safe sequence in seq. */
int is_safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;
    memcpy(work, avail, sizeof work);
    while (count < P) {
        int found = 0;
        for (int p = 0; p < P; p++) {
            if (finish[p]) continue;
            // TODO 1: If need = maxm[p] - alloc[p] is <= work for every resource, let p
            //   finish: add alloc[p] to work, mark finish[p], append p to seq and set
            //   found = 1
        }
        if (!found) return 0;
    }
    return 1;
}
void request(int p, const int req[R])
{
    for (int r = 0; r < R; r++)
        if (req[r] > maxm[p][r] - alloc[p][r] || req[r] > avail[r]) {
            printf("P%d request: must wait (not available)\n", p);
            return;
        }
    for (int r = 0; r < R; r++) { avail[r] -= req[r]; alloc[p][r] += req[r]; }
    int seq[P];
    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    } else {
        for (int r = 0; r < R; r++) { avail[r] += req[r]; alloc[p][r] -= req[r]; }
        printf("P%d request: denied (unsafe)\n", p);
    }
}
int main(void)
{
    int seq[P];
    if (is_safe(seq)) {
        printf("initial state safe:");
```

Expected output:

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

Show by hand why the request of P0 for (0, 2, 0) leads to an unsafe state.

Alternative Concept and Complexity Analysis

If deadlock were handled by detection and recovery (running a detection algorithm every few minutes and aborting a job) instead of the Banker's algorithm, what would the solution be? Evaluate the time per request ($O(m \times n^2)$ for every Banker's check versus no check), the cost of detection, and the work lost when a job is aborted.

Evidence Expected

A correct resource-allocation graph identifying all four conditions.

B. *Compare the Strategies – 3 Marks*

Evaluate prevention (removing one condition), avoidance (Banker's algorithm), detection and recovery, and ignoring deadlock (the ostrich approach) for the cluster.

Evidence Expected

A reasoned comparison of the four strategies for this environment.

C. *Apply the Banker's Algorithm – 4 Marks*

Implement the safety algorithm and the resource-request algorithm. Find a safe sequence for the given state and decide whether each request can be granted.

Evidence Expected

A working implementation with correct safe sequences and decisions.

D. *Predict Before Implementing – 1 Mark*

Predict how the running time of the Banker's algorithm grows with the number of processes n and resource types m.

Evidence Expected

A clear, testable prediction.

E. *Measure – 1 Mark*

Measure the time of the safety algorithm for 10, 100 and 1,000 processes and compare it with your prediction.

Evidence Expected

Measured times compared with the prediction.

F. *Defend the Final Design – 2 Marks*

Recommend the deadlock strategy for the research centre and justify it, including the cost of the Banker's algorithm and the need to know maximum demands in advance.

Evidence Expected

A critical, evidence-based recommendation.

A. Characterise deadlock (2 marks)

Necessary condition	In the research cluster
Mutual exclusion	A GPU or a licence can be used by one job at a time
Hold and wait	A job keeps its GPU while it waits for a simulation licence
No preemption	A GPU or licence cannot be taken from a running job without losing its work
Circular wait	J1 holds the GPU and waits for the licence; J2 holds the licence and waits for the GPU

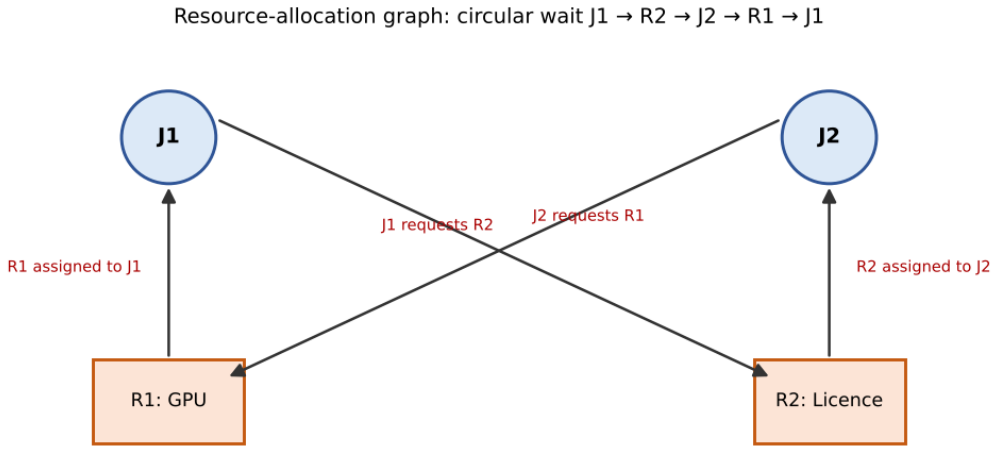


Figure 3 – Resource-allocation graph with a cycle: J1 → R2 → J2 → R1 → J1 (each resource has one instance, so the cycle means deadlock)

Request edges point from a job to a resource (J1 → R2, J2 → R1) and assignment edges from a resource to a job (R1 → J1, R2 → J2). All four conditions hold at the same time; removing any one of them prevents deadlock.

Complete the Code – TODO 1 (safety algorithm)

```
int ok = 1;
for (int r = 0; r < R; r++)
    if (maxm[p][r] - alloc[p][r] > work[r]) { ok = 0; break; }   /* need > work */
if (ok) {
```

```
    for (int r = 0; r < R; r++) work[r] += alloc[p][r];      /* p finishes, returns resources */
    finish[p] = 1; seq[count++] = p; found = 1;
  }
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

Output matches the expected output.

Hand verification of the safe sequence and of P0's request

Need = Max − Allocation: P0 (7,4,3), P1 (1,2,2), P2 (6,0,0), P3 (0,1,1), P4 (4,3,1); Available = (3,3,2). Work = (3,3,2): P1 can finish → (5,3,2); P3 → (7,4,3); P4 → (7,4,5); P0 → (7,5,5); P2 → (10,5,7). Safe sequence P1, P3, P4, P0, P2 – as printed.

After P1's request (1,0,2) is granted: Available = (2,3,0), Allocation of P1 = (3,0,2). **P4's request (3,3,0):** 3 > available 2 in the first resource → must wait.

P0's request (0,2,0): (0,2,0) ≤ Need P0 (7,4,3) and ≤ Available (2,3,0), so pretend to grant: Available = (2,1,0); Allocation P0 = (0,3,0); Need P0 = (7,2,3). Needs now: P1 (0,2,0), P2 (6,0,0), P3 (0,1,1), P4 (4,3,1). With Work = (2,1,0): P1 needs 2 of the second resource but only 1 is free ✗; P2 needs 6 of the first ✗; P3 needs 1 of the third but 0 is free ✗; P4 needs 4 of the first ✗; P0 needs 7 ✗. No process can finish, so the state is **unsafe** and the request is denied (rolled back). The state is not yet deadlocked – it is only that deadlock can no longer be ruled out.

Alternative Concept and Complexity Analysis – detection and recovery instead of the Banker's algorithm

Criterion	Banker's algorithm (avoidance)	Detection and recovery
Time per request	A safety check on every request: **O(m × n²)** (n processes, m resource types)	**None** on the request path (grant if free): O(m)
Cost of detection	Included in every request	Detection algorithm (same O(m × n²)) every few minutes: amortised cost per request ≈ 0; deadlock may exist for minutes before it is found (idle GPUs)
Work lost	None – deadlock never occurs; some requests are delayed	A job is aborted (or rolled back) – all computation since its last checkpoint is lost, possibly hours; repeat deadlocks possible
Information needed	Maximum demand of every job in advance	None

B. Compare the strategies (3 marks)

Strategy	How	Strengths	Weaknesses for the cluster
Prevention	Remove one condition: e.g., request all resources at once (no hold-and-wait), or number resources and request in order (no circular wait)	Simple, no run-time checks, guarantee	Low utilisation (GPUs held idle while the licence is waited for); ordering hard to impose on all users
Avoidance (Banker's)	Grant a request only if the state stays safe	Highest utilisation of the safe strategies; no aborts	Needs maximum demands in advance, fixed number of resources and processes; $O(m\ n^2)$ per request; conservative
Detection and recovery	Allow deadlock; periodically look for a cycle; abort or roll back jobs	No advance knowledge; no overhead on requests	Lost work; cluster stops until detected; recovery policy (which victim) needed
Ignore (ostrich)	Assume deadlock is rare	Zero cost; used by Linux/Windows for user resources	**Unacceptable here:** deadlocks already happen and the whole cluster stops until the administrator intervenes

C. Apply the Banker's algorithm (4 marks)

The safety algorithm and the resource-request algorithm were implemented as in the program above (file banker.c; compiled with -std=c11 -Wall -Wextra, no warnings):

- **Safety algorithm:** Work = Available; repeat: find an unfinished process whose Need ≤ Work, add its Allocation to Work and mark it finished; if none can be found, the state is safe only if all are finished.
- **Resource-request algorithm:** if Request > Need → error (exceeds maximum claim); if Request > Available → wait; otherwise pretend to allocate (Available −= Request, Allocation += Request, Need −= Request) and run the safety algorithm; if safe grant, else restore the old state and make the process wait.

Request	Decision	Reason
P1 (1,0,2)	**Granted** – safe sequence P1 P3 P4 P0 P2	≤ Need (1,2,2), ≤ Available (3,3,2); new state is safe
P4 (3,3,0)	**Must wait**	Available (2,3,0) < request in resource 0
P0 (0,2,0)	**Denied** – unsafe	Within need and available, but no process could finish afterwards (hand calculation above)

D. Predict before implementing (1 mark)

Prediction: the safety algorithm scans up to n processes in each of up to n passes and compares m resources per process, so the worst case is **$O(m \times n^2)$**; if processes finish in scan

order (best case) it needs one pass: O(m × n). With m fixed at 3, time should grow ≈ 100× when n grows 10× in the worst case, and ≈ 10× in the best case.

E. Measure (1 mark)

Average time of one safety check (compiled with -O2, m = 3). Two cases: a random state in which processes finish early in the first pass (typical), and a constructed worst case in which only the last process can finish first, then the one before it, and so on (n passes).

n	Random state (µs)	Worst case (µs)	Worst-case growth	Prediction O(n²) for worst case
10	0.09	0.12	–	–
100	0.64	8.41	× 70	× 100
1,000	6.08	597.5	× 71	× 100
3,000	19.9	5,184	× 8.7 (from n = 1,000)	× 9

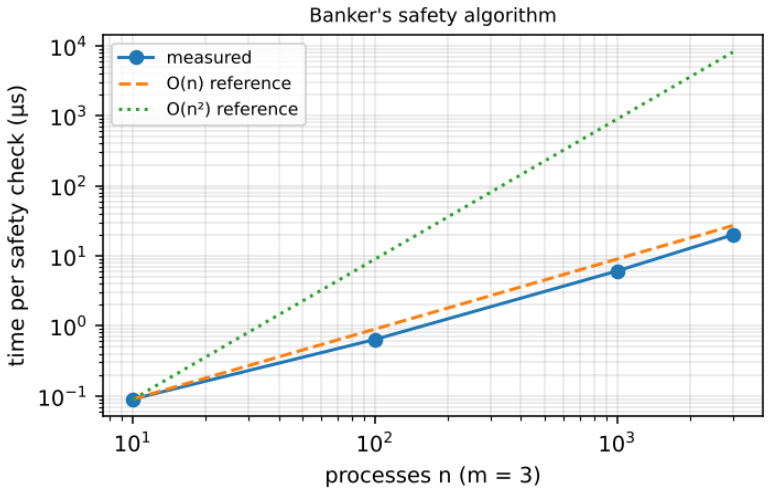


Figure 4 – Time of one safety check against n (random state; reference lines O(n) and O(n²))

Comparison: the random-state curve follows O(n) (×7–10 per 10× increase) – the first pass already finds most processes. The constructed worst case follows O(n²) (×70 per 10× from n = 10 to 100 and from 100 to 1,000, ×8.7 for 3× from 1,000 to 3,000, against predicted ×100 and ×9); the factor below 100 is due to cache effects and fixed per-call overhead for small n. The prediction O(m × n²) for the worst case is confirmed, and the best case O(m × n) is also confirmed.

F. Defend the final design (2 marks)

Recommendation: avoidance by the cluster scheduler, using the Banker's algorithm over the GPUs, licences and storage bandwidth, with detection as a safety net. Jobs submitted to a batch scheduler already declare their resource needs; the scheduler grants a request only if the state remains safe, so deadlock cannot occur and no work is lost.

Cost is small: the check costs ≈ 0.6 ms at n = 1,000 jobs even in the constructed worst case (5 ms at n = 3,000) – negligible against jobs that run for hours. The scheduler checks only when a job asks for resources, not on every instruction.

- **Why not detection and recovery:** aborting a long simulation wastes hours of GPU time; and between the deadlock and its detection the resources are idle.
- **Why not prevention alone:** requesting everything at once would leave expensive GPUs idle while jobs wait for a licence; but 'all-or-nothing' allocation is a useful simple fallback for small jobs.
- **Why not ignore:** the cluster already stops until an administrator intervenes.

Limitations: the Banker's algorithm needs the **maximum demand declared in advance** – users who overstate it lower utilisation (it is conservative), and users who understate it break the guarantee, so the scheduler must kill a job that exceeds its declared maximum. It also assumes a fixed set of resources and a fixed number of jobs; in a changing cluster the tables must be updated whenever a GPU is added or removed. Therefore a timeout-based detector (run every few minutes) is kept as a second line of defence.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Processes and process states	K5 – Evaluate	6
Q2	CPU scheduling	K5 – Evaluate	8
Q3	Threads	K5 – Evaluate	6

Question	Assessed Area	Bloom's Level	Marks
Q4	Inter-process communication	K5 – Evaluate	6
Q5	Synchronisation: critical sections, semaphores and monitors	K5 – Evaluate	11
Q6	Day-4 Design Challenge – Deadlock handling and Banker's algorithm	K5 – Evaluate	13
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your assignment report containing the required evidence for Q1–Q6
- C source code for all programs
- Gantt charts, diagrams, outputs and timings used as evidence

Assessment Verification

Students may be asked to explain any part of their design, code, calculations or results during a brief verification discussion/viva.

Overall Assessment Expectation

> **Model → Design → Implement → Measure → Compare → Evaluate → Recommend**